CUSTOMER SEGMENTATION & SERVICE RECOMMENDATION SYSTEM

Concepts • Methodology • Graphs • Results • Code • Viva Preparation

Project type: Machine Learning / Data Analytics demonstration project

Purpose: explain the complete workflow from customer data preparation through clustering, interpretation, service recommendations, and dashboard delivery.

Dataset disclosure: the included 1,200 customer records are synthetic demo data. They are not actual company/customer records and cannot establish real-world business impact.

1. Abstract

Businesses often serve customers whose needs, spending levels, engagement, and service experiences differ. Treating all customers identically can make communication less relevant. This project demonstrates a data-driven workflow that groups customers with similar numerical characteristics using K-Means clustering and then attaches understandable, rule-based service suggestions. The solution includes data preparation, feature scaling, cluster-count evaluation, visualization, profile summaries, exported customer-level results, and a Streamlit dashboard. It is an academic prototype built on synthetic data; its purpose is to demonstrate methods and reproducible engineering, not to claim verified revenue or retention improvements.

2. Introduction and Background

Customer segmentation is the practice of dividing a broad customer population into smaller groups that share meaningful characteristics. Traditional segmentation may use manually selected rules, such as spend bands or geography. Machine-learning segmentation can discover combinations of patterns across multiple variables, including activity, tenure, satisfaction, and spending. Unsupervised learning is appropriate when the dataset has no pre-existing target label that defines the correct segment. The algorithm finds structure in the selected features, while a human or business rule layer interprets the resulting groups.

3. Problem Statement

The project addresses the challenge of understanding different customer behaviour patterns in a single dataset and converting that understanding into practical service suggestions. Without segmentation, a business may send the same offer to highly engaged customers, new customers, low-satisfaction customers, and customers who frequently request help. The prototype aims to create a repeatable process for summarizing behaviour, identifying groups, and proposing differentiated actions. It does not automatically prove which action will work; that requires real-world testing.

4. Aim, Objectives, and Scope

The aim is to build an end-to-end customer segmentation and service recommendation prototype. Objectives are: (1) inspect and validate customer records; (2) prepare numeric features and handle missing values; (3) scale features so different units are comparable; (4) train K-Means models for candidate cluster counts; (5) compare inertia and silhouette score; (6) visualize customer groups; (7) summarize each group; (8) assign interpretable segment names and rule-based recommendations; (9) export data, metrics, plots, and model artifacts; and (10) expose results through a Streamlit dashboard. The scope excludes production deployment, real-time integration, and verified business impact.

5. Functional and Non-Functional Requirements

Functional requirements include loading a CSV, checking expected features, preprocessing values, evaluating candidate clusters, creating customer-level cluster assignments, generating profiles and recommendations, saving outputs, and displaying results in a dashboard. Non-functional requirements include reproducibility, understandable outputs, maintainable code, reasonable runtime on a typical laptop, clear error messages, and privacy-aware data handling. A production version would additionally require authentication, access control, logging, monitoring, model versioning, and formal data governance.

6. Technology Stack

Python orchestrates the workflow. Pandas and NumPy support data manipulation and numerical operations. Scikit-learn provides preprocessing, scaling, K-Means, PCA, and evaluation utilities. Matplotlib/Seaborn are used to produce graphs. Joblib stores a trained estimator or related artifact. Streamlit provides the interactive web interface. Jupyter Notebook supports exploration and learning. Requirements are recorded in requirements.txt so dependencies can be installed consistently. Exact library versions should be pinned for a formal reproducible deployment.

7. Dataset Description and Data Dictionary

The supplied CSV contains 1,200 synthetic records. The main numerical variables are age, website_visits_30d, monthly_spend_inr, service_inquiries_90d, engagement_score, tenure_months, satisfaction_score, and courses_enrolled. These features represent customer profile, activity, monetary value, service demand, relationship duration, satisfaction, and product adoption. Other descriptive columns such as customer identifiers, region, or preferred channel may support filtering

and interpretation, but only the features selected by the pipeline influence K-Means distances. The data dictionary in the project README and notebook should be treated as the authoritative list of columns for the exact version included in the ZIP.

8. Data Quality and Preprocessing

Before modelling, the pipeline should check column names, data types, duplicate records, missing values, invalid ranges, and whether numeric features can be converted safely. Missing values may be imputed using the median because the median is less sensitive to extreme values than the mean. Any imputation rule should be fitted on training data only when a formal train/test workflow exists. Customer identifiers should not be included as numeric predictors: an ID is a label, not a meaningful distance measure. Sensitive attributes should not be used without a legitimate, reviewed reason.

9. Exploratory Data Analysis (EDA)

EDA uses descriptive statistics and charts to understand distributions and relationships before clustering. Histograms reveal skewness and unusual ranges; box plots help identify outliers; correlation matrices show linear associations; count plots summarize categories; scatter plots show pairwise relationships. EDA helps discover data issues and informs interpretation, but correlation does not establish causation. Outliers should not be removed automatically: a high-spending customer may be a valid business case. Any exclusion or transformation should be justified and documented.

10. Feature Selection and Scaling

K-Means uses distances between points, so feature scale strongly affects results. Monthly spend measured in thousands can dominate a satisfaction score measured on a small scale if the features are not standardized. StandardScaler transforms each feature approximately to zero mean and unit variance using (x - mean) / standard deviation. This puts features on comparable scales, although it does not make the data normally distributed and can be sensitive to outliers. Feature selection should be based on relevance, availability, quality, and business meaning. Scaling parameters should be reused at inference time rather than recalculated inconsistently.

11. K-Means Clustering: Theory

K-Means partitions observations into k clusters. It initializes k centroids, assigns each observation to its nearest centroid, updates each centroid to the mean of its assigned observations, and repeats until assignments stabilize or a stopping condition is reached. Its objective is to minimize within-cluster sum of squared distances, commonly called inertia. K-Means is efficient and interpretable for numeric, scaled data, but it favours roughly spherical clusters, can be affected by outliers, and may converge to different local solutions. Multiple initializations and a fixed random_state improve reproducibility.

12. Choosing the Number of Clusters

The number k is not usually known in advance. The elbow method plots inertia for candidate k values. Inertia always tends to decrease as k increases, so the goal is to look for diminishing returns rather than the absolute minimum. The silhouette score compares how close a sample is to its own cluster versus the nearest alternative cluster; it ranges from -1 to +1, with larger values generally indicating better separation. Neither metric alone determines the business answer. Cluster interpretability, stability, size, actionability, and stakeholder needs should also be considered.

13. PCA and Cluster Visualization

Principal Component Analysis (PCA) transforms correlated numeric variables into orthogonal components that capture descending amounts of variance. A two-component projection makes it possible to display many-dimensional records on a 2D chart. However, information is lost when dimensions are reduced; visible overlap in two dimensions does not necessarily mean the full-dimensional clusters are identical. PCA is used here primarily for visualization, not as proof that the clusters are objectively correct. Axis labels should identify the components and any explained-variance percentages if available.

14. Cluster Evaluation Results

The saved evaluation reports the following silhouette values for candidate k: k=2 approximately 0.4028; k=3 approximately 0.3830; k=4 approximately 0.3009; k=5 approximately 0.2853; k=6 approximately 0.2911; k=7 approximately 0.2121; and k=8 approximately 0.2111. The k=2 score is highest among these tested values. The current pipeline is configured to use k=4 to

provide more granular business profiles. This is a deliberate trade-off, not evidence that k=4 is the metric-optimal choice. A formal final model decision should document why the extra granularity is useful and ideally compare stability and profile actionability.

15. Cluster Profiles and Interpretation

A cluster profile aggregates the mean or other summaries of each feature within a group. In the saved run, cluster 0 has 423 records, lower average spend and relatively low website visits; cluster 1 has 338 records, high average visits, high spend, high engagement, and longer tenure; cluster 2 has 182 records and the highest average service inquiries, with lower average satisfaction; cluster 3 has 257 records, older average age, lower visits, and moderate spend. These are descriptive summaries of synthetic data. Cluster numbers are arbitrary and may switch between runs; they are not rankings or permanent customer identities. Business labels should be assigned from profiles and clearly documented.

16. Cluster IDs vs Segment Labels

A numeric cluster_id is produced by K-Means. It indicates which centroid a record is nearest to in the fitted model. A customer_segment label is often created separately using readable rules such as “high engagement” or “support needed.” These fields are not inherently identical. For example, cluster 1 does not automatically mean “premium customer” unless a deliberate mapping defines it. The code should preserve both columns and explain the mapping logic so dashboard users do not confuse an algorithmic identifier with a business interpretation.

17. Service Recommendation Logic

The current recommendation layer uses explicit business rules rather than a separately trained recommender. It may suggest onboarding for new customers, an introductory service for customers with low product adoption, a loyalty or advanced service for highly engaged customers, or a proactive support follow-up for customers with low satisfaction or many inquiries. Rule ordering matters because multiple conditions may be true for one record. Recommendations should include a rationale where possible, be reviewed for fairness and relevance, and be evaluated against outcomes. A suggestion is not proof that the customer will accept it.

18. System Architecture and File Structure

The pipeline reads data from data/customer_data.csv, preprocesses selected features, fits and evaluates clustering configurations, produces customer assignments and profile summaries, and writes results under outputs/. Graphs are saved as image files and the fitted model is saved as an artifact. The notebook provides a learning and analysis workflow. The Streamlit application loads generated outputs to present key indicators, visualizations, tables, filters, and downloads. Keeping the data, code, outputs, and interface separated makes the project easier to test and maintain.

19. Graph Interpretation Guide

The elbow graph is used to compare inertia across k values; its curve normally declines as k grows. The silhouette graph compares separation quality across the tested values. The cluster scatter plot displays a 2D projection of the customer records coloured by cluster. Cluster profile tables are equally important because a visually attractive scatter plot does not tell a business what action to take. Each graph should have a descriptive title, axis labels, units, and a caption explaining what conclusion is supported and what cannot be inferred.

20. Dashboard Features and User Flow

The Streamlit dashboard makes results easier to inspect without reading source code. A typical user opens the app, reviews summary KPIs, chooses a segment filter, inspects cluster profiles and charts, searches for a customer record if supported, and exports filtered recommendations. The dashboard is a local demonstration interface, not a secured production application. Before deployment, authentication, authorization, validation of user inputs, safe handling of files, error logging, and restrictions on customer data exports would be necessary.

21. Testing and Validation Plan

Unit tests should verify that the CSV loads, required columns exist, imputation produces no missing numeric values, scaling returns expected dimensions, clustering creates a label for every row, profile counts sum to the total, recommendation output is non-empty where required, and exported files can be reopened. Integration tests should run the complete pipeline from input to outputs. Dashboard tests should cover missing files, empty filters, and invalid selections. Model validation should compare candidate k values, check cluster sizes, assess stability across seeds or samples, and inspect whether the profiles are useful to a human reviewer.

22. Installation and Execution

Create a virtual environment, activate it, install requirements, run the pipeline, and launch Streamlit. On Windows PowerShell: python -m venv .venv; .venv\\Scripts\\Activate.ps1; pip install -r requirements.txt; python run_pipeline.py; streamlit run app.py. If PowerShell blocks activation, follow the machine's approved policy or use Command Prompt activation (.venv\\Scripts\\activate.bat). If a dependency fails, check the Python version and package compatibility. Always run commands from the project root so relative file paths resolve correctly.

23. Limitations and Risks

The dataset is synthetic and cannot support claims about real customers, revenue, conversion, churn, or satisfaction improvements. K-Means assumes distance-based numeric structure and can be affected by outliers and scaling. The four-cluster configuration does not have the best silhouette score among the tested candidates. Rules are hand-designed and may be overly simplistic. The dashboard is not hardened for public deployment. If real customer data is introduced, data minimization, lawful access, consent where applicable, retention controls, security, fairness review, and a clear purpose for each feature are essential.

24. Future Enhancements

Possible extensions include evaluating Gaussian Mixture Models, hierarchical clustering, DBSCAN, or alternative mixed-data methods; incorporating categorical features through appropriate encoding; adding cluster stability analysis; building an API; adding automated tests and CI; pinning dependencies; versioning models and datasets; monitoring feature drift; and creating an experiment framework for recommendation effectiveness. A more advanced recommender could use historical service interactions, but only if reliable outcome data and an appropriate evaluation design are available.

25. Business KPIs and A/B Testing

Potential KPIs include recommendation click-through rate, service inquiry resolution time, course/service adoption, repeat engagement, satisfaction, retention, and cost per conversion. These should be measured with definitions and a baseline. To evaluate a recommendation, compare a treatment group receiving the recommendation with a suitable control group, using a pre-specified outcome and sufficient sample size. Do not infer uplift from cluster profiles alone. Any experiment must respect privacy and applicable business/ethical constraints.

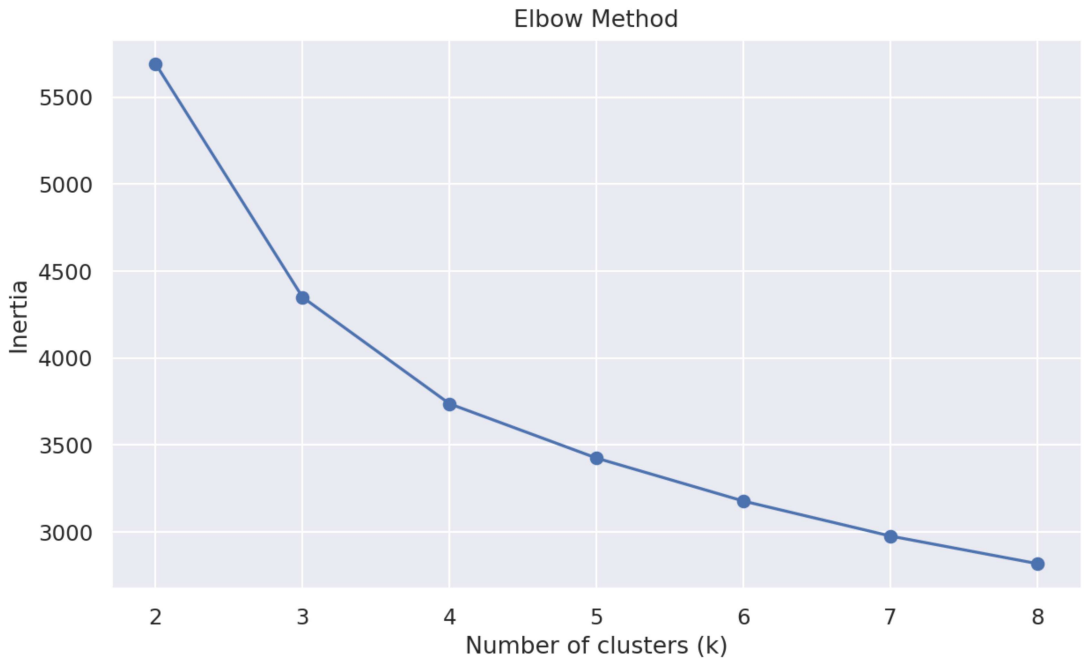
26. Conclusion

The project demonstrates a complete learning workflow from data preparation and exploratory analysis through unsupervised segmentation, evaluation, profile interpretation, rule-based suggestions, visualizations, and dashboard delivery. It combines code and explanations to support review and viva preparation. The most important interpretation is that clustering creates candidate groups, not unquestionable truths. Results should be checked for stability and actionability, and any business recommendation should be validated on approved real data before operational use.

27. Graphs and Actual Results

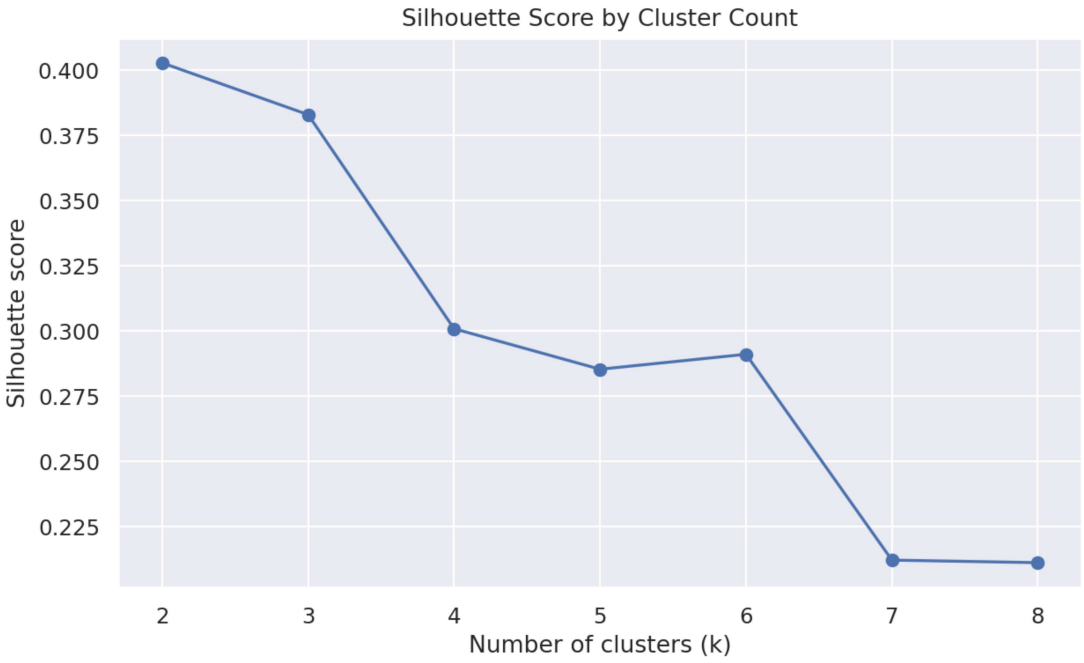
The graphs below are copied from the project output directory. They correspond to the saved run included in the package.

Figure 1. Elbow method



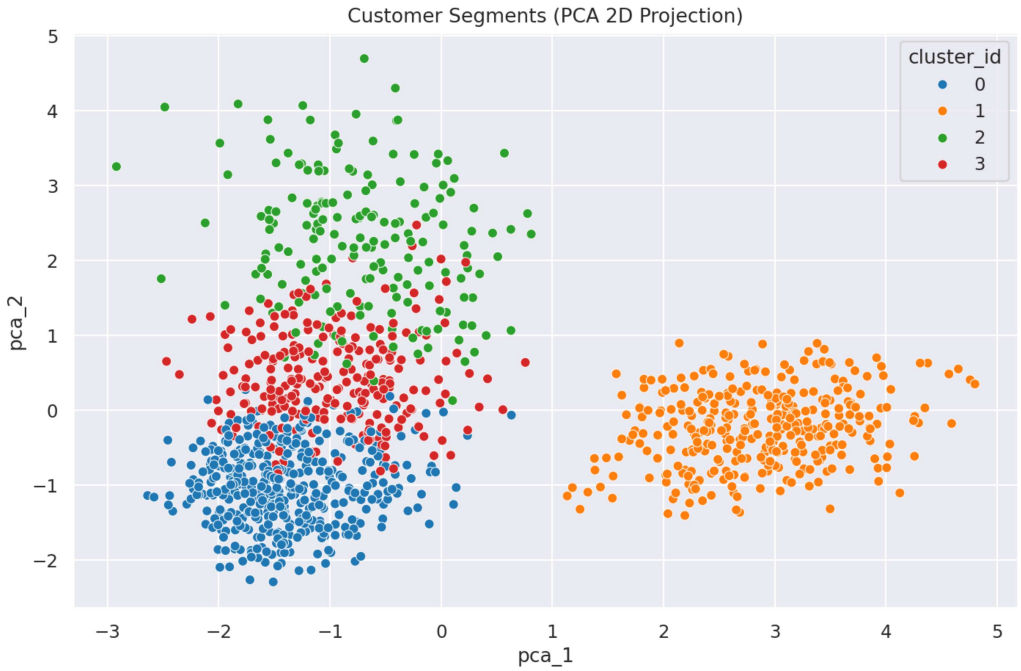
Read the decline in inertia and look for diminishing returns; there is no single universally correct elbow.

Figure 2. Silhouette score



The saved results show the highest tested silhouette score at k=2.

Figure 3. Customer cluster projection



This chart shows a low-dimensional view of clusters and should be interpreted with the numeric profiles.

Cluster Metrics Table

k	inertia	silhouette
2	5691.9316	0.4028
3	4349.5967	0.383
4	3737.0559	0.3009
5	3424.7833	0.2853
6	3177.5485	0.2911
7	2976.5086	0.2121
8	2817.7865	0.2111

Cluster Profile Table

cluster_id	customer_count	avg_age	avg_visits	avg_monthly_spend_inr	avg_service_inquiries	avg_engagement	avg_tenure_months	avg_satisfaction	avg_courses
0	423	28.89	4.23	3473.78	1.23	47.04	9.12	3.51	0.66
1	338	34.97	22.49	18778.22	0.99	87.19	42.2	4.0	3.55
2	182	44.4	9.45	6228.3	9.87	55.47	33.6	2.87	1.36
3	257	47.49	5.45	4921.8	1.8	42.15	27.81	3.51	1.05

Appendix A — Pipeline Source Code

Complete source listing from run_pipeline.py. Code is split into sections only for page layout; the ZIP contains the original executable file.

```
from pathlib import Path
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

ROOT = Path(__file__).resolve().parent
DATA_PATH = ROOT / "data" / "customer_data.csv"
OUT = ROOT / "outputs"
OUT.mkdir(exist_ok=True)

NUMERIC_FEATURES = [
    "age", "website_visits_30d", "monthly_spend_inr",
    "service_inquiries_90d", "engagement_score", "tenure_months",
    "satisfaction_score", "courses_enrolled"
]
CATEGORICAL_FEATURES = ["preferred_channel", "region"]

def segment_name(row, medians):
    if row["engagement_score"] >= medians["engagement_score"] and row["website_visits_30d"] >= medians["website_visits_30d"]:
        return "Highly Engaged Customers"
    if row["service_inquiries_90d"] >= medians["service_inquiries_90d"] and row["service_inquiries_90d"] >= 5:
        return "Service-Focused Customers"
    if row["tenure_months"] <= 5:
        return "New Customers"
    return "Occasional Users"

def recommend(row):
    recs = []
    if row["engagement_score"] >= 75 and row["website_visits_30d"] >= 12:
        recs.append("Advanced learning program or premium service bundle")
    elif row["tenure_months"] <= 5:
        recs.append("New-customer onboarding and introductory learning path")
    elif row["service_inquiries_90d"] >= 5:
        recs.append("Guided consultation, support clinic, or service walkthrough")
    elif row["website_visits_30d"] <= 5:
        recs.append("Re-engagement campaign with a short, relevant learning module")
    else:
        recs.append("Personalized course recommendation and monthly service package")
    if row["courses_enrolled"] == 0:
        recs.append("Beginner-friendly starter course")
    if row["satisfaction_score"] <= 3:
        recs.append("Follow-up satisfaction check and proactive support")
    return " | ".join(recs)

def main():
    df = pd.read_csv(DATA_PATH)
    df["monthly_spend_inr"] = pd.to_numeric(df["monthly_spend_inr"], errors="coerce")
    num_imputer = SimpleImputer(strategy="median")
    X_num = pd.DataFrame(num_imputer.fit_transform(df[NUMERIC_FEATURES]), columns=NUMERIC_FEATURES, index=df.index)
    cat_imputer = SimpleImputer(strategy="most_frequent")
    X_cat = pd.DataFrame(cat_imputer.fit_transform(df[CATEGORICAL_FEATURES]), columns=CATEGORICAL_FEATURES, index=df.index)
    medians = X_num.median()
    X_scaled = StandardScaler().fit_transform(X_num)

    # Compare candidate cluster counts with inertia and silhouette score.
    metrics = []
    for k in range(2, 9):
        model = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = model.fit_predict(X_scaled)
        metrics.append({"k": k, "inertia": float(model.inertia_), "silhouette": float(silhouette_score(X_scaled, labels))})
    pd.DataFrame(metrics).to_csv(OUT / "cluster_metrics.csv", index=False)

    # Four clusters are used as a practical starting point; tune after reviewing metrics.
    kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    pca = PCA(n_components=2, random_state=42)
    coords = pca.fit_transform(X_scaled)

    result = df.copy()
    for col in NUMERIC_FEATURES:
        result[col] = X_num[col]
    for col in CATEGORICAL_FEATURES:
        result[col] = X_cat[col]
    result["cluster_id"] = labels
    result["pca_1"] = coords[:, 0]
    result["pca_2"] = coords[:, 1]
    result["customer_segment"] = result.apply(lambda r: segment_name(r, medians), axis=1)
    result["service_recommendation"] = result.apply(recommend, axis=1)
    result.to_csv(OUT / "segmented_customers.csv", index=False)

    profile = result.groupby("cluster_id").agg(
        customer_count=("customer_id", "count"),
        avg_age=("age", "mean"),
        avg_visits=("website_visits_30d", "mean"),
```

```
        avg_monthly_spend_inr=("monthly_spend_inr", "mean"),
        avg_service_inquiries=("service_inquiries_90d", "mean"),
        avg_engagement=("engagement_score", "mean"),
        avg_tenure_months=("tenure_months", "mean"),
        avg_satisfaction=("satisfaction_score", "mean"),
        avg_courses=("courses_enrolled", "mean")
    ).round(2).reset_index()
    profile.to_csv(OUT / "cluster_profiles.csv", index=False)

    sns.set_theme()
    plt.figure(figsize=(8, 5))
    m = pd.DataFrame(metrics)
    plt.plot(m["k"], m["inertia"], marker="o")
    plt.xlabel("Number of clusters (k)")
    plt.ylabel("Inertia")
    plt.title("Elbow Method")
    plt.tight_layout()
    plt.savefig(OUT / "elbow_method.png", dpi=160)
    plt.close()

    plt.figure(figsize=(8, 5))
    plt.plot(m["k"], m["silhouette"], marker="o")
    plt.xlabel("Number of clusters (k)")
    plt.ylabel("Silhouette score")
    plt.title("Silhouette Score by Cluster Count")
    plt.tight_layout()
    plt.savefig(OUT / "silhouette_scores.png", dpi=160)
    plt.close()

    plt.figure(figsize=(9, 6))
    sns.scatterplot(data=result, x="pca_1", y="pca_2", hue="cluster_id", palette="tab10", s=35)
    plt.title("Customer Segments (PCA 2D Projection)")
    plt.tight_layout()
    plt.savefig(OUT / "customer_clusters.png", dpi=160)
    plt.close()

    summary = {
        "rows": int(len(result)),
        "features_used": NUMERIC_FEATURES,
        "selected_k": 4,
        "best_silhouette_k": int(m.loc[m["silhouette"].idxmax(), "k"]),
        "best_silhouette_score": round(float(m["silhouette"].max()), 4),
        "note": "Synthetic demonstration data; replace with approved real customer data before business use."
    }
    (OUT / "run_summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
    joblib.dump({"kmeans": kmeans, "scaler": StandardScaler().fit(X_num), "numeric_features": NUMERIC_FEATURES}, OUT / "clustering_model.joblib")
    print("Pipeline complete. See the outputs/ folder.")

if __name__ == "__main__":
    main()
```

Appendix B — Streamlit Dashboard Source Code

Complete source listing from app.py. Code is split into sections only for page layout; the ZIP contains the original executable file.

```
from pathlib import Path
import pandas as pd
import streamlit as st

ROOT = Path(__file__).resolve().parent
OUTPUT = ROOT / "outputs" / "segmented_customers.csv"
DATA = ROOT / "data" / "customer_data.csv"

st.set_page_config(page_title="Customer Segmentation & Service Recommendations", page_icon="■", layout="wide")
st.title("Customer Segmentation & Service Recommendation System")
st.caption("Synthetic demonstration data — validate recommendations before operational use.")

if OUTPUT.exists():
    df = pd.read_csv(OUTPUT)
else:
    st.warning("Processed outputs not found. Run `python run_pipeline.py` first.")
    df = pd.read_csv(DATA)

if "customer_segment" not in df.columns:
    st.info("Showing raw sample data until the pipeline is run.")
    st.dataframe(df.head(20), use_container_width=True)
    st.stop()

c1, c2, c3, c4 = st.columns(4)
c1.metric("Customers", f"{len(df):,}")
c2.metric("Segments", df["cluster_id"].nunique())
c3.metric("Average monthly spend", f"■{df['monthly_spend_inr'].mean():,.0f}")
c4.metric("Average engagement", f"{df['engagement_score'].mean():.1f}/100")

st.subheader("Customer segment overview")
segment_counts = df["customer_segment"].value_counts().rename_axis("segment").reset_index(name="customers")
st.bar_chart(segment_counts.set_index("segment"))

left, right = st.columns(2)
with left:
    st.subheader("Cluster profiles")
    profile_path = ROOT / "outputs" / "cluster_profiles.csv"
    if profile_path.exists():
        st.dataframe(pd.read_csv(profile_path), use_container_width=True)
with right:
    st.subheader("2D cluster visualization")
    chart_path = ROOT / "outputs" / "customer_clusters.png"
    if chart_path.exists():
        st.image(str(chart_path), use_container_width=True)
    else:
        st.info("Run the pipeline to create the cluster visualization.")

st.subheader("Find customers and recommendations")
segments = ["All"] + sorted(df["customer_segment"].dropna().unique().tolist())
selected = st.selectbox("Filter by segment", segments)
search = st.text_input("Search customer ID")
view = df.copy()
if selected != "All":
    view = view[view["customer_segment"] == selected]
if search.strip():
    view = view[view["customer_id"].astype(str).str.contains(search.strip(), case=False, na=False)]
columns = ["customer_id", "customer_segment", "cluster_id", "engagement_score", "website_visits_30d", "monthly_spend_inr", "service_inquiries_90d"
st.dataframe(view[columns], use_container_width=True, hide_index=True)

csv = view.to_csv(index=False).encode("utf-8")
st.download_button("Download filtered customer results (CSV)", csv, "customer_recommendations.csv", "text/csv")
```

Viva Questions and Suggested Answers

Q. What is customer segmentation?

Answer: It divides customers into groups with similar characteristics to support analysis and tailored actions.

Q. Why use unsupervised learning?

Answer: The input has no predefined target label for the correct customer segment, so the algorithm discovers structure.

Q. Why K-Means?

Answer: It is relatively simple, scalable, and easy to explain for numeric scaled data, although it has assumptions and limitations.

Q. Why scale features?

Answer: Distance-based algorithms can be dominated by variables with larger numeric ranges if features are not scaled.

Q. What is inertia?

Answer: The sum of squared distances from observations to their assigned cluster centroids.

Q. What does silhouette score measure?

Answer: How similar an observation is to its own cluster compared with the nearest alternative cluster; higher is generally better.

Q. Why use k=4 when k=2 scores higher?

Answer: The demo uses four clusters for more detailed profiles; this is a business-granularity trade-off that needs validation, not a claim that k=4 has the best silhouette.

Q. What is PCA?

Answer: A dimensionality-reduction technique used here to project many features into two dimensions for visualization.

Q. Is the recommender machine learning?

Answer: The current recommendation layer is rule-based; K-Means is the machine-learning component.

Q. Can the results prove business improvement?

Answer: No. The dataset is synthetic. Real impact requires approved real data and controlled evaluation.

Q. What is a limitation of K-Means?

Answer: It is sensitive to scaling and outliers and works best for roughly spherical clusters in numeric feature space.

Q. How would you improve this project?

Answer: Test cluster stability, compare algorithms, add tests, validate recommendations with experiments, and implement production security and monitoring.

Customer Segmentation & Service Recommendation System | Python, Pandas, Scikit-learn, K-Means, PCA, Streamlit
Built an end-to-end analytics prototype that preprocesses customer features, compares K-Means cluster configurations using inertia and silhouette score, profiles customer groups, generates rule-based service suggestions, exports model outputs and visualizations, and presents findings in a Streamlit dashboard. Used synthetic data for demonstration; business impact was not claimed.